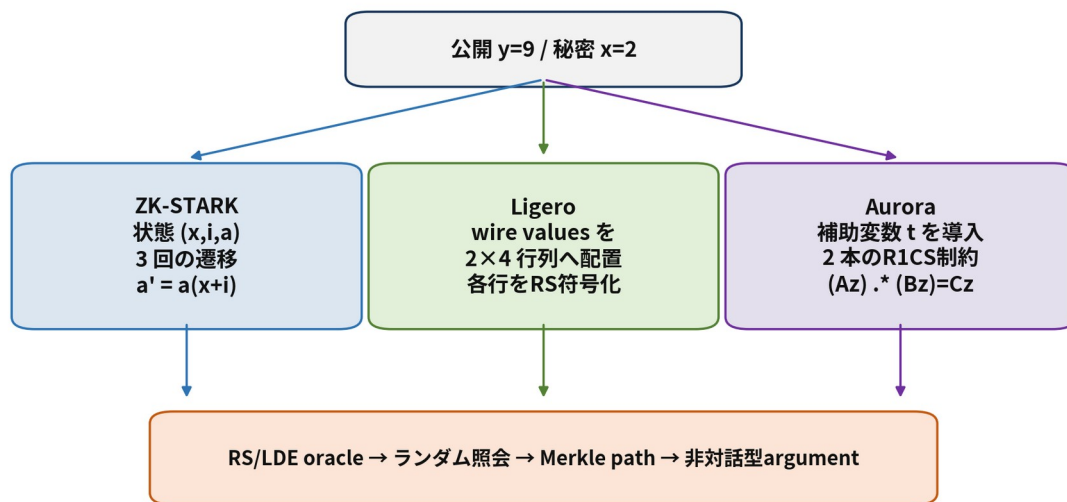


透明・ハッシュベース汎用 ZKP 三方式の 読解補助・数学的背景解説書

ZK-STARK・Ligero・Aurora を、有限体・符号・IOP・零知識化から読み直す

同じ関係 $y=(x+1)(x+2)(x+3)$ を三つの表現へ写す



本解説書の中心図: 同じ relation を三方式が別々に arithmetize する

対象: 「透明・ハッシュベース汎用 ZKP 三方式の設計論」統合解説論文

2026 年 7 月

本書の位置づけ

本書は、先に作成した統合解説論文を「読める」だけでなく、「数式を自力で再構成し、実装上の意味まで説明できる」状態へ導くための別冊である。ZKP 一般の定義、完全性・健全性・零知識性の初歩は繰り返さず、三方式で繰り返し現れる難所に焦点を当てる。

主要な方針は三つである。第一に、有限体上の多項式、Reed-Solomon 符号、IOP、Merkle commitment を一つの共通言語で整理する。第二に、ZK-STARK、Ligerio、Aurora の方式固有部分を、同じ toy relation に対する異なる arithmetization として対比する。第三に、F₁₇ 上の数値を逐一検算し、抽象的な定理と具体的な protocol transcript の間を埋める。

対象読者 統合論文を一読したが、vanishing polynomial、LDE、proximity、FRI、interleaved RS、univariate sumcheck、randomized encoding の相互関係を整理したい読者を想定する。

証拠の扱い 漸近量は各原論文の定理・表に基づき、実測値は測定条件を併記する。F₁₇ の数値例は本書で機械検算した教育用計算であり、暗号学的安全性を示すものではない。

推奨する三つの読み方

読み方	順序	到達点
代数優先	第2章 → 第3章 → 第5章 → 第7/10/12章	多項式恒等式と符号距離から各 protocol を導出できる
protocol 優先	第1章 → 第4章 → 第6/9/11章 → 第14章	各 message・challenge・query の役割を追跡できる
実装優先	第4章 → 第8/10/13章 → 第15章	field、FFT、Merkle、memory、statement model の設計判断ができる

表記上の約束

記号	意味
F	有限体。toy では F ₁₇ を用いる
H	message、trace、constraint を置く domain
L/D	LDE または codeword を評価する、通常は H より大きい domain
$Z_H(X)$	H 上でゼロとなる monic polynomial $\prod_{h \in H} (X-h)$
$RS[L, \rho]$	L 上で次数 $< \rho L $ の多項式を評価した Reed-Solomon 符

記号	意味
	号
$\Delta(w,C)$	word w と code C の相対 Hamming 距離
N, s, T	それぞれ R1CS/回路規模、回路規模、実行 step 数を表す 文脈依存量

目次

PART I 共通数学とプロトコル基盤	7
1. 三方式の読解地図	8
2. 有限体と多項式の必須事項	10
2.1 F_{17} を使う意味と限界	10
2.2 subgroup、coset、domain	11
2.3 補間と評価	11
2.4 消失多項式と factor theorem	12
2.5 Schwartz-Zippel 型の乱数圧縮	12
3. Reed-Solomon 符号と proximity	13
3.1 LDE はなぜ必要か	14
3.2 membership と proximity の違い	14
3.3 interleaving	14
4. IOP から非対話型 argument へ	15
4.1 IPCP、IOP、IOPP	15
4.2 Merkle binding が保証するもの	16
4.3 Fiat-Shamir の順序	16
5. 零知識化を統一的に理解する	17
5.1 bounded independence	17
5.2 blinding codeword	17
5.3 random self-reducibility	17
PART II ZK-STARK の難所を解く	19
6. AIR と execution trace	20
6.1 trace 設計はコンパイラ設計である	20
6.2 APR と一般 RAM	21
7. composition polynomial と FRI	22
7.1 constraint quotient	22
7.2 FRI folding の代数	22
7.3 FRI が証明するものと証明しないもの	23
8. ZK-STARK の実装・応用上の補足	24
PART III Liger の難所を解く	25
9. witness 行列と interleaved RS	26
9.1 interleaved test	27
10. linear ・ quadratic test と零知識化	28
10.1 linear constraints	28
10.2 quadratic constraints	28
10.3 randomized encoding と blind codeword	28
10.4 batching	29
PART IV Aurora の難所を解く	30
11. R1CS から rowcheck ・ lincheck へ	31
11.1 toy R1CS 行列	31
11.2 rowcheck	32

11.3 lincheck	32
12. 単変数 sumcheck と generic compiler	33
12.1 なぜ通常の multivariate sumcheck を使わないか	33
12.2 multiplicative coset 版の恒等式	33
12.3 additive subspace 版	34
12.4 amortization	34
12.5 RS-encoded IOP compiler	34
12.6 zero knowledge self-reduction	34
13. Aurora の実装・応用上の補足	35
PART V 三方式を横断して設計判断する	36
14. F_17 toy computation の横断解剖	37
14.1 toy が忠実に示すもの	38
14.2 toy が省略するもの	38
15. エビデンスに基づく比較と方式選択	39
15.1 information-theoretic IOP の漸近比較	39
15.2 実測値が食い違う理由	40
15.3 selection matrix	40
16. soundness と security review の実務	41
17. よくある難所と誤解の解消	42
18. 結論	43
付録 A. F_17 arithmetic と domain 一覧	44
付録 B. toy computation 完全データ	45
B.1 STARK	45
B.2 Ligerio	45
B.3 Aurora	46
付録 C. 用語集	47
付録 D. 参考文献とエビデンス注記	48

本書の視覚ガイド

図は証明そのものではなく、定義・帰着・乱数の流れを固定するための概念図である。表中の「proof length」は IOP oracle 全体の長さとして最終 non-interactive argument size を区別する。比較表の数値は歴史的実装のスナップショットであり、現在の実装性能を直接代表しない。

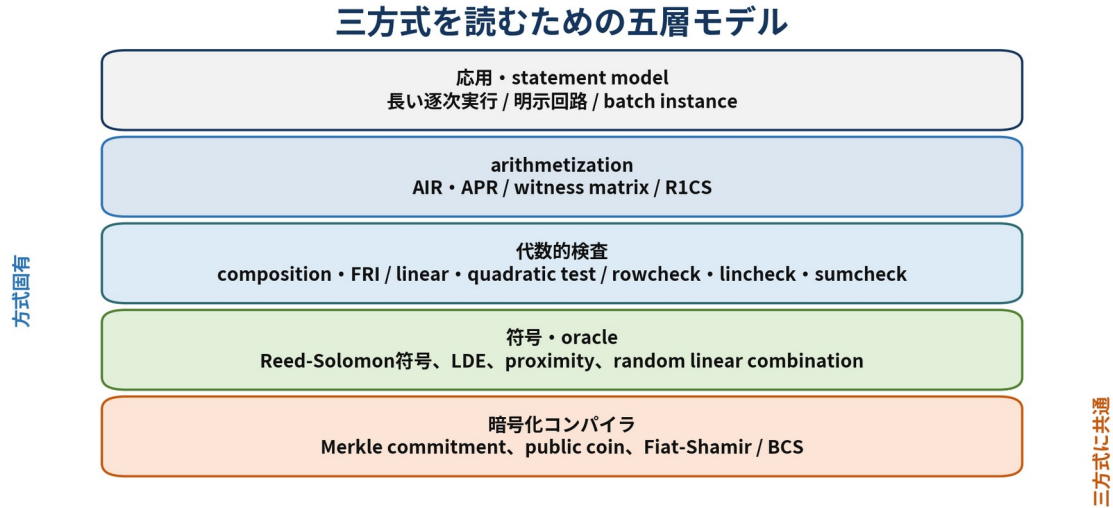


図 1 三方式を読むための五層モデル

本書作成。

PART I

共通数学とプロトコル基盤

有限体、多項式、符号、IOP、Merkle、零知識化を一つの体系として整理する。

1. 三方式の解地

統合論文との対応 統合論文 §1-2、および各方式の導入節。

この章で得るもの 方式固有の arithmetization と、共通 backend を分離して理解する。

ZK-STARK、Ligerio、Aurora は、いずれも「巨大な計算整合性を、少数のランダム照会で検査できる符号語へ変換する」という方針を共有する。しかし、何を最初の代数的対象とするかが異なる。ZK-STARK は実行 trace、Ligerio は wire values の行列、Aurora は明示 R1CS である。

この差は単なる記法ではない。statement description の長さ、verifier が必ず読む情報、prover の memory access、proof size の漸近形、batching の容易さを決める。したがって「三方式のどれが最良か」という問いは、relation をどのモデルで与えるかを固定しなければ意味を持たない。

同じ関係 $y=(x+1)(x+2)(x+3)$ を三つの表現へ写す

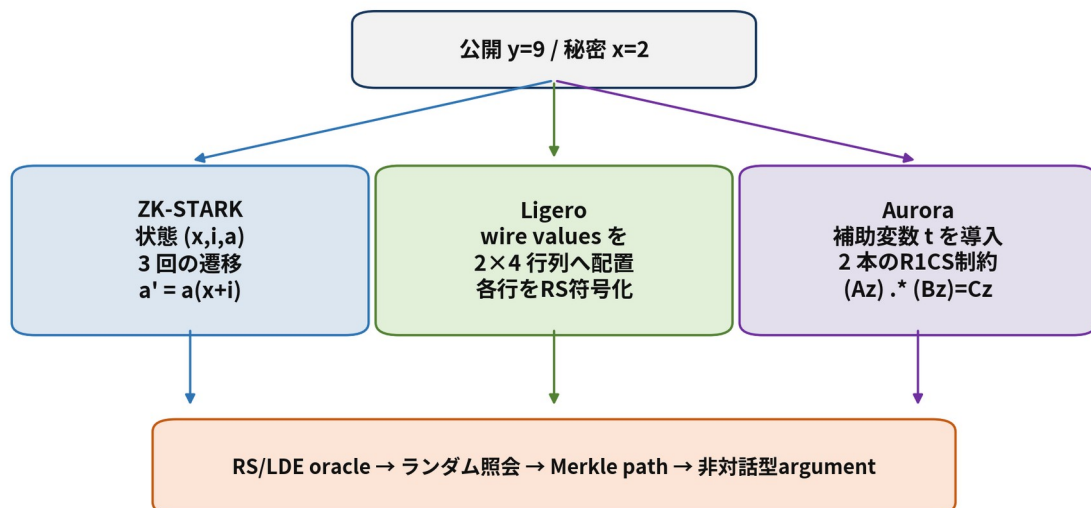


図 2 同じ toy relation に対する三つの arithmetization

本書作成。

表 1 三方式の層別対応

層	ZK-STARK	Ligerio	Aurora
statement	program + time bound / machine execution	arithmetic or Boolean circuit	explicit R1CS matrices
witness 形状	$T \times w$ execution trace	$m \times \ell$ extended-witness matrix	z, Az, Bz, Cz の LDE
中心検査	constraint quotient + FRI	interleaved/linear/quadratic tests	rowcheck + lincheck + univariate sumcheck
低次数確認	FRI	集約 codeword を明示的に RS membership 検査	generic compiler + FRI

層	ZK-STARK	Ligerio	Aurora
典型的長所	短い program の長い逐次実行	単純性、平方根通信、batch	explicit R1CS で短い argument

最重要の区別 IOP の oracle proof length と、Merkle/Fiat-Shamir 後の最終 argument size は別量である。Aurora の IOP は $O(N)$ 長でも、最終 proof は query 数と authentication path 数により $O(\log^2 N)$ となる。

2. 有限体と多項式の必須事項

統合論文との対応 統合論文 § 2.1-2.2、付録 A。

この章で得るもの F_{17} の domain、補間、消失多項式、coset、次数を自力で計算できる。

2.1 F_{17} を使う意味と限界

F_{17} は 17 個の元を持つ素体であり、加減乗除は整数演算を 17 で剰余化すればよい。非零元 F_{17}^* は位数 16 の巡回群であるため、位数 2、4、8、16 の乗法部分群を持つ。toy computation では、trace domain $H=\{1,4,16,13\}$ と、位数 8 の domain $D=\{1,2,4,8,16,15,13,9\}$ を利用する。

一方、 F_{17} は暗号用途には小さすぎる。ランダム線形結合による失敗確率 $1/|F|$ は $1/17$ であり、FRI や sampling を数回行っても十分な security bit を得られない。toy の目的は、各式を手で追えることにある。

表 2 F_{17} の乗法逆元

a	$a^{-1} \bmod 17$	確認
1	1	$1 \cdot 1 \equiv 1$
2	9	$2 \cdot 9 \equiv 1$
3	6	$3 \cdot 6 \equiv 1$
4	13	$4 \cdot 13 \equiv 1$
5	7	$5 \cdot 7 \equiv 1$
6	3	$6 \cdot 3 \equiv 1$
7	5	$7 \cdot 5 \equiv 1$
8	15	$8 \cdot 15 \equiv 1$
9	2	$9 \cdot 2 \equiv 1$
10	12	$10 \cdot 12 \equiv 1$
11	14	$11 \cdot 14 \equiv 1$
12	10	$12 \cdot 10 \equiv 1$
13	4	$13 \cdot 4 \equiv 1$
14	11	$14 \cdot 11 \equiv 1$
15	8	$15 \cdot 8 \equiv 1$

a	$a^{-1} \bmod 17$	確認
16	16	$16 \cdot 16 \equiv 1$

2.2 subgroup、coset、domain

乗法部分群 H は積と逆元で閉じ、各元のべき和が強い消去性を持つ。位数 m の部分群 H に対して、 $0 < k < m$ なら $\sum_{h \in H} h^k = 0$ である。この恒等式が、Aurora の multiplicative-coset 版 sumcheck で「総和が定数項だけで決まる」理由である。

coset $gH = \{gh : h \in H\}$ は H を乗法的に平行移動した集合である。message domain と evaluation domain を disjoint にしたい場合、 H と交わらない coset を選ぶ。Aurora toy の $L=3D$ は H と交わらず、randomized LDE の外部照会を一様化する。

注意 2018 ZK-STARK 原論文の実装は二進拡大体と加法 affine subspace を中心に構成される。本書の F_{17} toy FRI は、計算を小さく保つため乗法 domain 版として説明する。概念は共通するが、実装 domain は同一ではない。

2.3 補間と評

n 個の相異なる点 $h_0, \dots, h_{n-1}$ と値 $v_0, \dots, v_{n-1}$ を与えると、次数 $< n$ の多項式 f が一意に存在する。これは Lagrange 補間である。逆に f を domain 上で評価すれば値ベクトルを得る。FFT/IFFT は、domain に群構造があるとき、この双方向変換を $O(n \log n)$ で行う。

値の表と低次数多項式は、domain を固定すれば同じ情報である



LDE: 同じ多項式を、より大きい domain L で評価する情報を増やすのではなく、冗長性と距離を導入する操作である

図 3 補間・評価・LDE の関係

本書作成。

$$f(X) = \sum_i v_i \cdot \prod_{j \neq i} (X - h_j) / (h_i - h_j) \text{ in } F[X] \quad (2.1)$$

2.4 消失多項式と factor theorem

$Z_H(X) = \prod_{h \in H} (X - h)$ は H の全点でゼロとなる。多項式 C が H 上でゼロであることと、 C が Z_H で割り切れることは同値である。これにより、 $|H|$ 個の局所等式を、一つの商多項式 $Q = C/Z_H$ の低次数性へ変換できる。

「H上でゼロ」を一つの多項式恒等式へ圧縮する

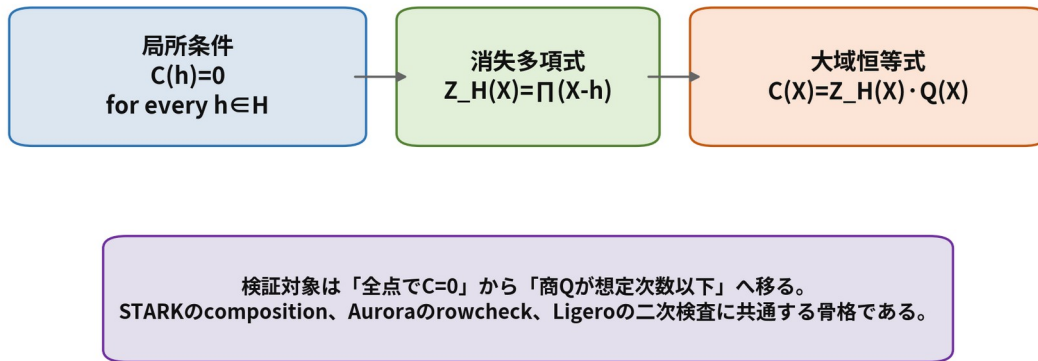


図4 消失多項式による局所条件の大域化

本書作成。

直観 Z_H は「 H 上でだけ消える共通因子」である。正しい constraint polynomial はこの因子を持つ。不正な constraint は通常この因子を持たず、商が多項式にならないか、想定 degree を超える。

2.5 Schwartz-Zippel 型の乱数圧縮

非零多項式 p の次数が d なら、ランダム $\alpha \in F$ で $p(\alpha) = 0$ となる確率は高々 $d/|F|$ である。lincheck では残差 vector e を power vector $(1, \alpha, \dots, \alpha^{m-1})$ で圧縮し、 $h(\alpha) = 0$ を検査する。random linear combination による constraint aggregation も同じ原理の線形版である。

$$\Pr_{\alpha \leftarrow F}[p(\alpha) = 0] \leq \deg(p)/|F| \quad \text{for nonzero } p \quad (2.2)$$

3. Reed-Solomon 符号と proximity

統合論文との対応 統合論文 § 2.2-2.3、§ 3.4、§ 4.2、§ 5.5。

この章で得るもの 「低次数性」と「誤り訂正距離」を同じ対象として理解する。

$RS[L, \rho]$ は、domain L 上で次数 $< \rho|L|$ の多項式を評価した word の集合である。線形 code であり、rate はおおむね ρ 、相対 minimum distance はおおむね $1-\rho$ である。低次数という代数的制約が、Hamming 空間での距離を生む。

Reed-Solomon 符号を「低次数多項式の評価集合」として見る

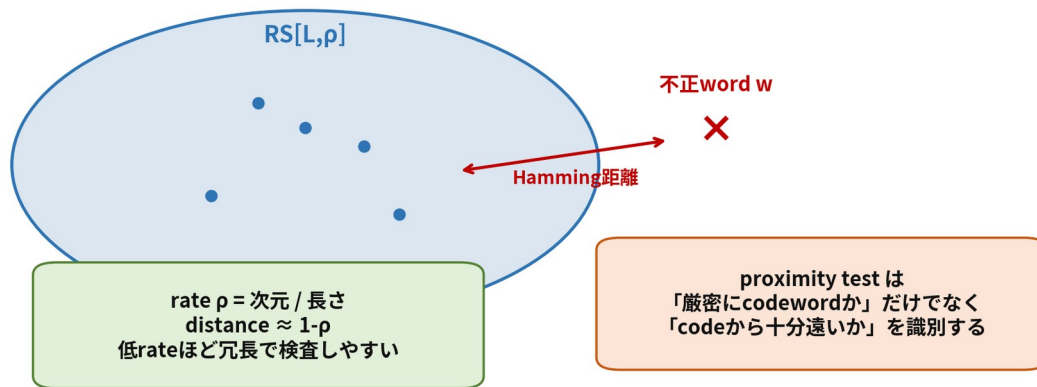


図 5 Reed-Solomon 符号と proximity の幾何学的直観

本書作成。

表 3 多項式表現と符号理論表現の対応

概念	多項式側	符号側
degree bound	$\deg f < k$	dimension k
evaluation domain	$ L =n$	block length n
redundancy	$n-k$ 個の余分な評価	distance $d=n-k+1$
不正性	高次数または整合しない値	code から Hamming 距離がある
検査	low-degree test / quotient	membership / proximity test

3.1 LDE はなぜ必要か

H 上の n 個の値は次数 $< n$ の多項式を一意に定めるが、H そのものだけを開示すると、値を直接読むことになる。より大きい L へ評価する LDE は、同じ情報を冗長な codeword へ変え、少数の誤りが全体の整合性と衝突するようにする。

重要なのは、LDE が新しい semantic information を追加しない点である。追加されるのは、局所照会から大域一貫性を推測するための冗長性である。

3.2 membership と proximity の違い

membership test は word が厳密に code に属するかを問う。proximity test は、word が code から δ 以上遠い場合に reject する。IOP では、悪意ある prover が完全に random な word を送るため、厳密 membership より「遠さ」を soundness 仮定へ組み込む方が自然である。

FRI は RS proximity を query-efficient に検査する。一方、Ligerio は長さ約 $\sqrt{s}$ の集約 codeword w を丸ごと受け取り、その RS membership を直接確認する。この設計差が、Ligerio の 2-round/simple protocol と $\sqrt{s}$ communication、Aurora/STARK の $\log$ rounds/polylog openings を分ける。

3.3 interleaving

m 本の codeword を行に並べた $m \times n$ 行列を interleaved codeword とみなすと、距離は列単位で測る。ある列に一つでも誤りがあれば、その列全体を error column と数える。Ligerio は random row combination で誤り列を一つの codeword へ投影し、少数列照会で整合性を確認する。

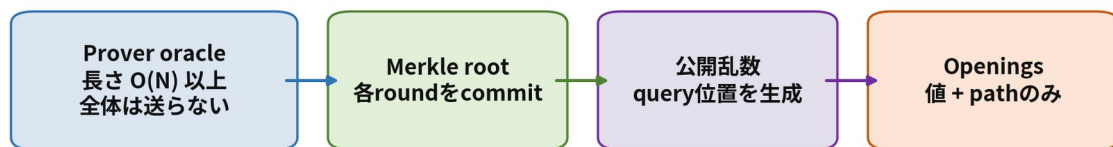
4. IOP から非対話型 argument へ

統合論文との対応 統合論文 § 2.3、§ 3.5、§ 4.5、§ 5.5。

この章で得るもの oracle、commitment、query、Fiat-Shamir の責務分離を理解する。

IOP では、prover message は巨大な oracle として存在し、verifier は少数位置だけを読む。現実の network では oracle access をそのまま実装できないため、prover は各 oracle を Merkle tree へ commit し、照会された leaf と authentication path だけを送る。

IOPの巨大oracleを、短いargumentとして持ち運ぶ



最終argument size $\approx$ commitments + queried symbols + authentication paths
したがって「IOP proof length $O(N)$ 」と「最終proofがpolylog」は両立する。

図 6 IOP oracle から短い argument への変換

本書作成。

$$\text{argument bytes} \approx \sum \text{roots} + q \cdot \text{field_bytes} + \sum \text{authentication_path_bytes} \quad (4.1)$$

4.1 IPCP、IOP、IOPP

表 4 oracle proof model の区別

モデル	prover message	verifier の仕事	本書での代表
IPCP	最初に一つの proof oracle、その後は通常 message	oracle query + interaction	Ligerio
IOP	複数 round で oracle message を送る	各 round 後に乱数を返し、最後に query	STARK, Aurora
IOPP	与えられた word が relation/code に近いか検査	witness oracle にも query	FRI, low-degree test

4.2 Merkle binding が保証するもの

Merkle root は oracle の内容を隠す commitment ではなく、主に binding を与える。leaf をそのまま hash するだけでは hiding は保証されない。零知識性は、oracle 値自体を randomized encoding で隠した上で、Merkle tree が後から値を変更できないようにすることで成立する。

4.3 Fiat-Shamir の順序

challenge は、それが検査する oracle の commitment 後に生成されなければならない。prover が challenge を知ってから oracle を選べると、random linear combination や query sampling の soundness が失われる。実装では transcript domain separation、serialization の一意性、round label が安全性の一部である。

1. oracle round 0 を構成し、Merkle root を transcript へ追加する。
2. hash から challenge ρ_0 を導出する。
3. ρ_0 に依存する oracle round 1 を構成し、root を追加する。
4. 必要 round を繰り返し、最後に query 位置を導出する。
5. query 値と Merkle path を開示し、verifier が全 local relation を再計算する。

post-quantum の注記 三論文は、公開鍵代数ではなく hash を中心に用いるため「plausibly post-quantum」と位置づける。ただし、non-interactive 化を量子 adversary まで含めて主張するには QROM 上の解析と十分な hash output 長が必要であり、「hash-based だから自動的に証明済み」と解釈すべきではない。

5. 零知識化を統一的に理解する

統合論文との対応 統合論文 § 2.4、§ 3.5、§ 4.5、§ 5.6。

この章で得るもの randomized LDE、blind codeword、self-reduction を一つの affine-space 原理で結ぶ。

H 上で message を固定する多項式は一意ではない。degree budget を増やすと、全解は $f_0 + Z_H R$ の形を持つ。R をランダムに選べば、H 上の意味は保ったまま、H 外の評価をランダム化できる。

同じmessageを符号化する多項式は一つではない

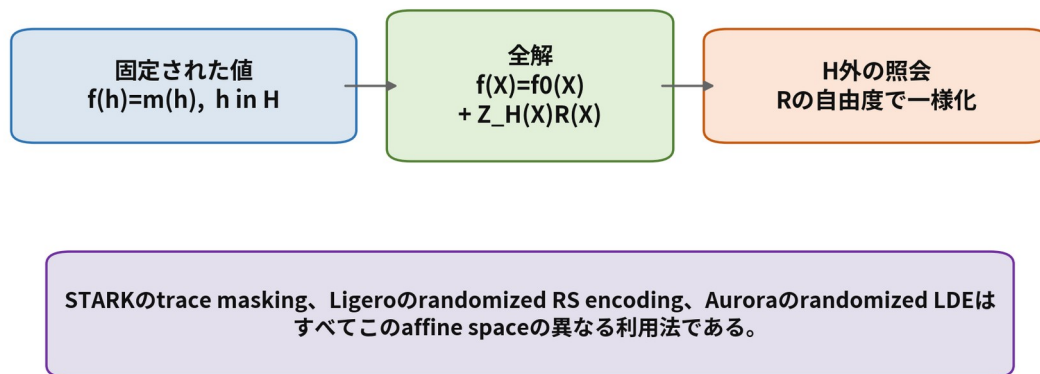


図 7 message を保つ多項式の affine family

本書作成。

5.1 bounded independence

degree を b だけ増やし、 b 個以下の H 外評価点を考えると、それらの値は適切な条件下で一様独立になる。線形代数的には、R の係数から照会値への evaluation map が full rank であることを使う。Aurora の「 b -query verifier に対する perfect ZK」はこの性質を明示的に利用する。

5.2 blinding codeword

Ligerio では verifier が $r^{\wedge}TU$ という codeword 全体を受け取るため、局所照会の randomization だけでは不十分である。そこで zero message に decode する random codeword u_0 を加え、 $r^{\wedge}TU + u_0$ を開示する。これは線形 combination そのものを one-time pad する操作である。

5.3 random self-reducibility

Aurora の sumcheck では $f = g + Z_H h$ と除算するため、 f に加えた $Z_H R$ mask が h 側へ吸収され、remainder g から消えることがある。そこで random codeword q を先に送り、challenge c 後に

cf+q の sum を証明する。sumcheck 入力自体を一様な codeword へ変換するため、division 後も秘密に依存しない。

表 5 三方式の零知識機構

方式	局所照会を隠す	大域 message を隠す	追加の注意
ZK-STARK	trace polynomial + Z_{HR}	composition にも random mask/constraint randomization	実装により masking detail が異なる
Ligerio	randomized RS encoding	zero-message blind codeword	affine interleaved tests で soundness を維持
Aurora	degree-b randomized LDE	cf+q self-reduction	message/evaluation domains を disjoint にする

共通の誤解 「Merkle tree が witness を隠す」のではない。Merkle は commit 後の改変を防ぐ。witness を隠すのは、commit 前に作られた randomized oracle である。

PART II

ZK-STARK の難所を解く

AIR、constraint quotient、composition、FRI、memory/routing の位置づけを掘り下げる。

6. AIR と execution trace

統合論文との対応 統合論文 § 3.1-3.3、§ 3.8。

この章で得るもの 逐次計算を local transition polynomial へ変換し、width ・ degree ・ length の tradeoff を説明できる。

AIR は、各 step の machine state を有限体 vector として並べ、current state X と next state Y の組が低次数 polynomial constraints を満たすことを要求する。trace の列数 w が state width、行数 T が cycle count である。

STARK: 実行を「隣接行の低次数制約」に変換する

step	x	i	a
0	2	1	1
1	2	2	3
2	2	3	12
3	2	4	9

遷移制約
 $x' - x = 0$
 $i' - i - 1 = 0$
 $a' - a(x+i) = 0$

境界制約
 $i_0 = 1, a_0 = 1$
 $a_3 = y = 9$

図 8 F_17 toy relation の AIR trace

本書作成。

表 6 toy AIR の制約

制約種別	toy 式	役割
state persistence	$x' - x = 0$	秘密 x が step 間で変化しない
index transition	$i' - i - 1 = 0$	factor index を 1 ずつ増やす
accumulator	$a' - a(x+i) = 0$	積を逐次更新する
boundary	$i_0 = 1, a_0 = 1, a_3 = y$	初期値と公開 output を固定する

6.1 trace 設計はコンパイラ設計である

同じ計算でも、register を増やせば constraint degree を下げられ、register を減らせば degree が上がる。prover cost は概ね「trace length × width × FFT overhead」に支配され、composition

degree は FRI domain の blowup factor へ影響する。したがって AIR は単なる正しさの記述ではなく、性能を決める intermediate representation である。

設計則 高 degree の一つの constraint より、補助 register を追加して低 degree constraint へ分解した方が、LDE/FRI の総コストが下がる場合が多い。ただし width 増加と memory bandwidth の代償がある。

6.2 APR と一般 RAM

toy trace はすべての state が隣接 step だけで検証でき、非局所 memory access を持たない。一般 RAM では、同じ address への read/write が time 順に整合することを確認する必要がある。2018 論文は Algebraic Placement and Routing (APR) を用い、time-order trace と memory-order trace を関連付ける。

space-bounded computation では width S が小さいため、 $T \times S$ trace を直接扱える。一般 computation では memory consistency 用の permutation/routing が必要となり、prover overhead が追加される。この差が、STARK を explicit circuit へ単純適用したとき Aurora より log factor 重いという比較につながる。

7. composition polynomial と FRI

統合論文との対応 統合論文 § 3.3-3.4、§ 3.8。

この章で得るもの constraint aggregation と degree halving を、F_17 の数値で再現する。

7.1 constraint quotient

trace 列を補間した $X(T)$, $I(T)$, $A(T)$ に対し、active domain $H_{\text{active}}=\{1,4,16\}$ 上で遷移制約を課す。最終点 13 から next step は存在しないため、active domain から除く。

$$Z_{\text{active}}(T)=(T-1)(T-4)(T-16)=T^3+13T^2+16T+4 \quad (7.1)$$

$$C_i(T)=I(4T)-I(T)-1=4Z_{\text{active}}(T) \quad (7.2)$$

$$C_a(T)=A(4T)-A(T)(X(T)+I(T))=Z_{\text{active}}(T)(11+15T^2+10T^3) \quad (7.3)$$

各商は正しい trace なら低次数である。複数商を別々に FRI へ渡すと oracle と query が増えるため、commit 後に得た random coefficients で一つの composition D へ集約する。

$$D(T)=5 \cdot 4 + 7(11+15T^2+10T^3)=12+3T^2+2T^3 \quad (7.4)$$

STARK: 多数の制約を一つの低次数claimへ集約する

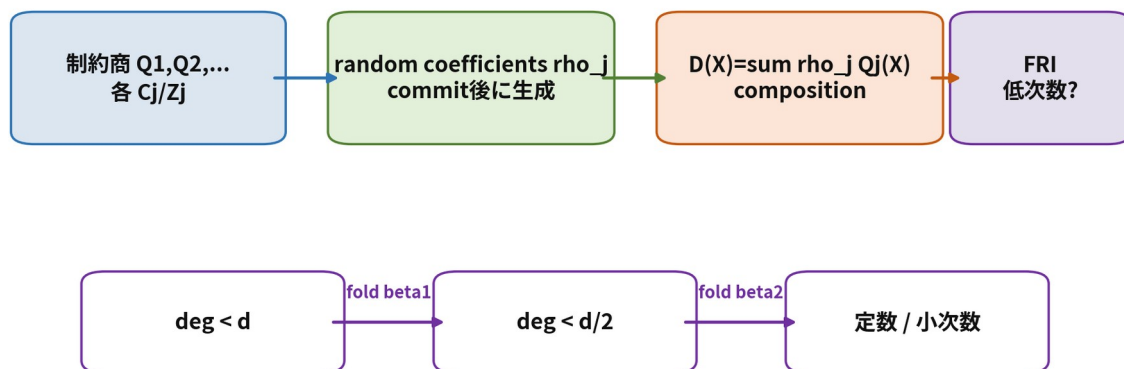


図 9 constraint quotient、composition、FRI の流れ

本書作成。

7.2 FRI folding の代数

multiplicative-domain 版では $f(X)=f_{\text{even}}(X^2)+X f_{\text{odd}}(X^2)$ と偶数次数・奇数次数へ分ける。challenge β に対し $f_{\text{next}}(Y)=f_{\text{even}}(Y)+\beta f_{\text{odd}}(Y)$ を作ると degree がほぼ半分になる。正しい低次数 f なら必ずこの関係が保たれる。

$$f_{\text{next}}(x^2) = (f(x)+f(-x))/2 + \beta \cdot (f(x)-f(-x))/(2x) \quad (7.5)$$

F_17 toy FRI: $D(X)=12+3X^2+2X^3$

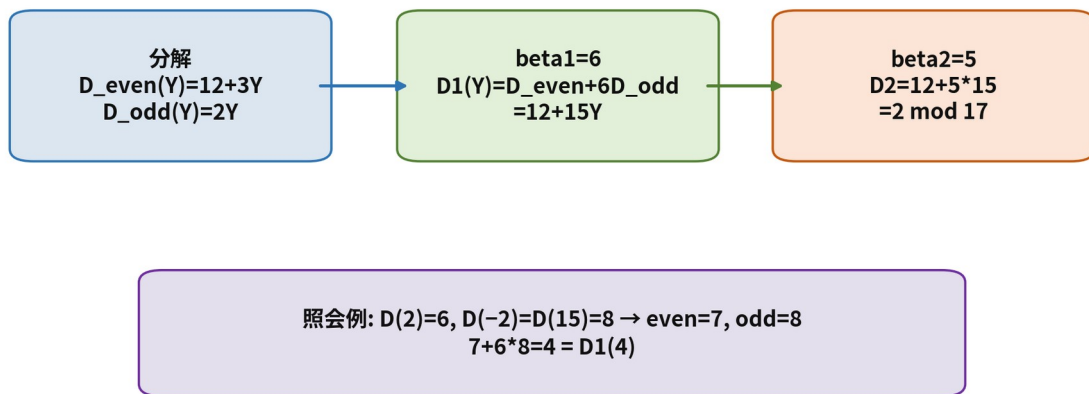


図 10 F_17 toy FRI の二回 fold

本書作成。

表 7 toy FRI の degree halving

round	polynomial	degree bound	challenge
0	$D(X)=12+3X^2+2X^3$	3	$\beta_1=6$
1	$D_1(Y)=12+15Y$	1	$\beta_2=5$
2	$D_2=2$	0	-

7.3 FRI が証明するものと証明しないもの

FRI は、提示された oracle がある低次数 RS codeword に近いことを示す。FRI だけでは、その codeword が元の execution trace や constraints に対応することは示さない。trace opening と composition identity を同じ query 位置で結び、境界条件を別に検査して初めて computational integrity となる。

よくある誤解 「FRI が計算を検証する」のではない。FRI は低次数性を検査する。計算の意味は AIR と composition identity が与え、Merkle commitment が同じ oracle への整合した照会を保証する。

8. ZK-STARK の実装・応用上の補足

統合論文との対応 統合論文 § 3.5-3.7、§ 7。

この章で得るもの field、FFT、hash、memory、statement model の選択が性能へ与える影響を整理する。

STARK prover は、trace generation 自体よりも、複数列の LDE、composition 評価、Merkle tree 構築、FRI oracle 生成に多くの memory bandwidth を使う。算術演算回数だけでなく、cache locality、streaming、parallel FFT、hash throughput が支配的になり得る。

表 8 STARK 実装の主要ボトルネック

設計項目	性能への影響	典型的な失敗
field	FFT domain、乗算 cost、hash/AIR 相性	便利な field を選んだ結果、constraint degree や extension cost が増える
trace width	LDE 列数、Merkle leaf 幅	補助 register を増やしすぎる
constraint degree	blowup factor、FRI rate	高 degree custom gate を無制限に使う
hash	Merkle throughput、proof size	field arithmetic と hash 変換の cost を無視する
memory layout	prover wall-clock、RAM peak	oracle 全体を常時保持する設計

応用適性が高いのは、短い program が長い逐次 state transition を生成する場合、長い VM execution、秘匿 database scan、ledger state transition などである。逆に、既に小さな explicit R1CS があり、verifier がその instance を読むことを受け入れるなら、Aurora の方が proof size と prover overhead で有利になり得る。

historical evidence 2018 論文の DPM proof-of-concept は、DNA profile 数 2^{20} 付近で naive verification に対する time ratio 約 1/6、witness-size ratio 約 1/100 を報告する。ただし専用 benchmark、当時の hardware、60-bit 設定に依存する実測であり、一般的性能値ではない[1, Sec. 1.1, Appendix A]。

PART III

Ligerio の難所を解く

平方根通信、interleaved RS、linear/quadratic tests、blinding を幾何と線形代数から理解する。

9. witness 行列と interleaved RS

統合論文との対応 統合論文 § 4.1-4.2、§ 4.8。

この章で得るもの 平方根通信の最適化と random row combination の soundness を説明できる。

Ligerio は extended witness の s 個の field elements を $m \times \ell$ 行列へ置き、各 row を長さ n の RS codeword へ拡張する。verifier へ送る集約 codeword の長さはおおむね ℓ に比例し、query で開く一列の大きさは m に比例する。 $m\ell \approx s$ の下で両者を均衡させると $m \approx \ell \approx \sqrt{s}$ となる。

Ligerio: witnessをほぼ正方形に配置する理由

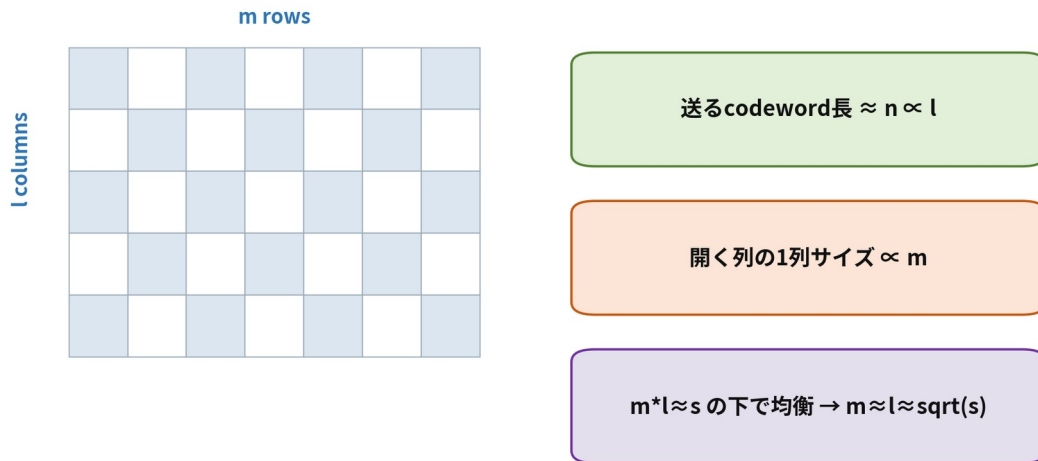


図 11 Ligerio の平方根 tradeoff

本書作成。

$$\text{communication} \approx a \cdot \ell + b \cdot t \cdot m, \text{ subject to } m\ell \approx s \quad (9.1)$$

厳密な最適値は field size、security repetition、Merkle path、linear/quadratic message 長に依存するが、平方根の漸近形はこの積制約から生じる。

9.1 interleaved test

Ligerioのinterleaved RS test

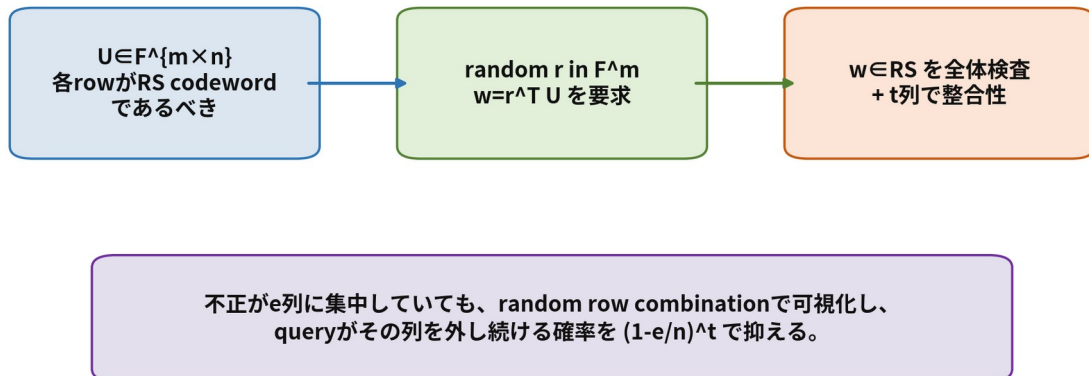


図 12 random row combination と column query

本書作成。

U が正しい interleaved codeword なら、任意の r に対して $w = r^T U$ も RS codeword であり、各列で $w_j = \sum_i r_i U_{i,j}$ が成り立つ。不正 U では、random combination が偶然 code へ近づく確率と、query set Q が error columns を外す確率を分けて評価する。

$$\Pr[\text{accept}] \leq \text{algebraic-collapse term} + (1-e/n)^t \quad (9.2)$$

2022 拡張版は RS code に対する解析改善を取り込み、 $e < d/2$ で algebraic term をおおむね $n/|F|$ とする。これは code の unique-decoding 半径に近い領域まで soundness 分析を広げるものである[2, Sec. 4.1.1, App. C]。

10. linear • quadratic test と零知識化

統合論文との対応 統合論文 § 4.3-4.5、§ 4.8。

この章で得るもの random aggregation、polynomial consistency、blind codeword の役割を数値例で追う。

10.1 linear constraints

$Ax=b$ が偽なら、residual $d=Ax-b$ は非零である。uniform r に対し $r^T d=0$ となる確率は $1/|F|$ である。Ligerio は $r^T A$ の係数を各 message block 上の polynomial $r_i(X)$ へ補間し、 $q(X)=\sum_i r_i(X)p_i(X)$ を送らせる。

toy では aggregate coefficient vector は $(13,4,2,7,4,0,3,0)$ であり、 $w=(1,2,3,4,5,12,9,0)$ との内積は $102 \equiv 0$ である。 2×4 に reshape すると、row 係数は $(13,4,2,7)$ と $(4,0,3,0)$ である。message points での q 値は $(16,8,16,11)$ で、総和は $51 \equiv 0$ となる。

$$\Pr[\text{false linear relation passes}] \leq 1/|F| + ((e+k+\ell)/n)^t \quad (10.1)$$

10.2 quadratic constraints

乗算制約を $x \cdot y + a \cdot z = b$ の形へ並べると、各 row polynomial の積により degree がほぼ $2k-2$ へ上がる。toy で X-message= $(3,12,0,0)$ 、Y-message= $(4,5,0,0)$ 、Z-message= $(12,9,0,0)$ を $\zeta=\{0,1,2,3\}$ 上に置くと、 $p_X p_Y - p_Z$ は Z_ζ で割り切れ、商は $3X^2+11X+5$ となる。

$$p_X(X)p_Y(X) - p_Z(X) = Z_\zeta(X)(3X^2+11X+5) \quad (10.2)$$

$$\Pr[\text{false quadratic relation passes}] \leq 1/|F| + ((e+2k)/n)^t \quad (10.3)$$

10.3 randomized encoding と blind codeword

toy の一行目 $p_1=1+X$ に対し、 $p_{\sim 1}=p_1+Z_\zeta(4+2X)$ を用いる。 ζ 上の message は不変であるが、 $\eta=\{4,\dots,11\}$ 上の codeword は $(4,3,4,15,0,0,16,11)$ となる。二行目も独立に randomize する。

$r=(3,5)$ と $u_0=Z_\zeta(6+3X)$ に対して $v=3p_{\sim 1}+5p_{\sim 2}+u_0$ を返すと、 η 上で $v=(8,5,14,9,12,8,13,6)$ である。 $\eta=7$ では $3 \cdot 15 + 5 \cdot 6 + 2 = 77 \equiv 9 = v(7)$ となる。

表 9 toy Ligerio の randomized codeword と blind combination

η	row 1	row 2	u_0	$v=3r_1+5r_2+u_0$
4	4	8	7	8
5	3	12	4	5
6	4	3	4	14
7	15	6	2	9

η	row 1	row 2	u_0	$v=3r_1+5r_2+u_0$
8	0	0	12	12
9	0	8	2	8
10	16	0	16	13
11	11	0	7	6

なぜ u_0 は zero message か u_0 が message points で 0 なら、verifier へ送る v の message-level relation は変えない。一方、evaluation points では random codeword として $r^T U$ の全体を隠す。

10.4 batching

同じ circuit に対する複数 instance では、係数 matrix や encoding infrastructure を共有し、random linear combination で instances を束ねられる。Ligerio の論文は $N=\Omega(\kappa^2)$ で amortized communication が circuit size を下回り、total verification が $O(s \log s + N \log N)$ field operations となる設定を示す[2, Sec. 5.4]。

historical evidence 拡張版は SHA-256 preimage、soundness 2^{-40} で約 35KB を例示する。また 2022 版の比較表では小中規模で短い proof を報告するが、field、statement 生成、hardware が異なるため Aurora/STARK の別論文数値と直接比較してはならない[2, Abstract, Sec. 6]。

PART IV

Aurora の難所を解く

R1CS、rowcheck、lincheck、単変数 sumcheck、RS-encoded IOP compiler を段階的に導出する。

11. R1CS から rowcheck ・ lincheck へ

統合論文との対応 統合論文 § 5.1-5.3、§ 5.9。

この章で得るもの toy relation の A, B, C 行列を構成し、三つの線形検査と一つの積検査へ分解する。

Aurora の statement は、 $A, B, C \in \mathbb{F}^{m \times (n+1)}$ 、public input v 、witness w である。 $z=(1, v, w)$ に対し $(Az) \cdot (Bz) = Cz$ を要求する。explicit matrix representation を読むだけで $\Omega(N)$ 時間が必要なため、verifier $O(N)$ はこの model では input-linear である。

Aurora: R1CSを線形部分と乗算部分へ分解する

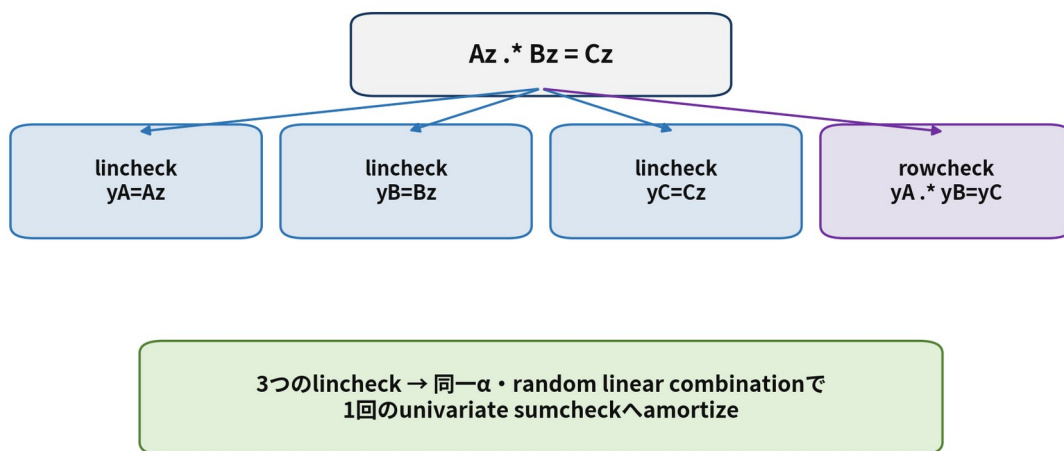


図 13 R1CS の三 lincheck + 一 rowcheck 分解

本書作成。

11.1 toy R1CS 行列

補助変数 $t=(x+1)(x+2)$ を導入し、 $z=(1, y, x, t)=(1, 9, 2, 12)$ とする。制約は $(x+1)(x+2)=t$ と $t(x+3)=y$ である。

$$A = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 & 1 & 0 \\ 3 & 0 & 1 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (11.1)$$

表 10 toy R1CS の評価

row	Az	Bz	Cz	確認
1	3	4	12	$3 \cdot 4 = 12$
2	12	5	9	$12 \cdot 5 = 60 \equiv 9$

11.2 rowcheck

constraint domain H_1 上で $f_A f_B - f_C = 0$ なら、numerator は $Z_{\{H_1\}}$ で割り切れる。Aurora では商を独立 oracle として送らず、ratio を virtual oracle として扱い、その rate を generic low-degree compiler へ渡すことができる。

$$q_{\text{row}}(X) = (f_A(X)f_B(X) - f_C(X)) / Z_{\{H_1\}}(X) \quad (11.2)$$

toy の masked polynomials では $q_{\text{row}} = 14 + 7X + 3X^2 + 4X^3 + 10X^5 + 5X^6$ であり、remainder は 0 である。mask により degree は上がるが、compiler が許容 rate を調整する。

11.3 lincheck

$x = My$ の残差 e を全座標で読む代わりに、 α から power vector $r(\alpha) = (1, \alpha, \dots, \alpha^{m-1})$ を生成し、 $h(\alpha) = \langle r, e \rangle$ を検査する。 $e \neq 0$ なら h は非零 polynomial であり、偶然 0 となる確率は $m/|F|$ 以下である。

toy の A-lincheck で $\alpha = 6$ を選ぶと $r = (1, 6, 2, 12)$ 、padding 後の $r^{\wedge}TA = (1, 0, 1, 6)$ である。

$r^{\wedge}T(Az) = 3 + 6 \cdot 12 = 7$ 、 $(r^{\wedge}TA)z = 1 + 2 + 6 \cdot 12 = 7$ となり一致する。

12. 単変数 sumcheck と generic compiler

統合論文との対応 統合論文 § 5.3-5.6、§ 5.9。

この章で得るもの sum の恒等式、division、amortization、FRI 接続、self-reduction を理解する。

12.1 なぜ通常の multivariate sumcheck を使わないか

古典 sumcheck は Reed-Muller code の tensor structure を利用し、変数を一つずつ固定する。Aurora の oracle は単変数 Reed-Solomon codeword であり、変数を減らす方向が存在しない。そこで、sum claim を特定係数の消失と低次数 subcode membership へ直接変換する。

Auroraの単変数sumcheck: sum claimを低次数subcodeへ変える

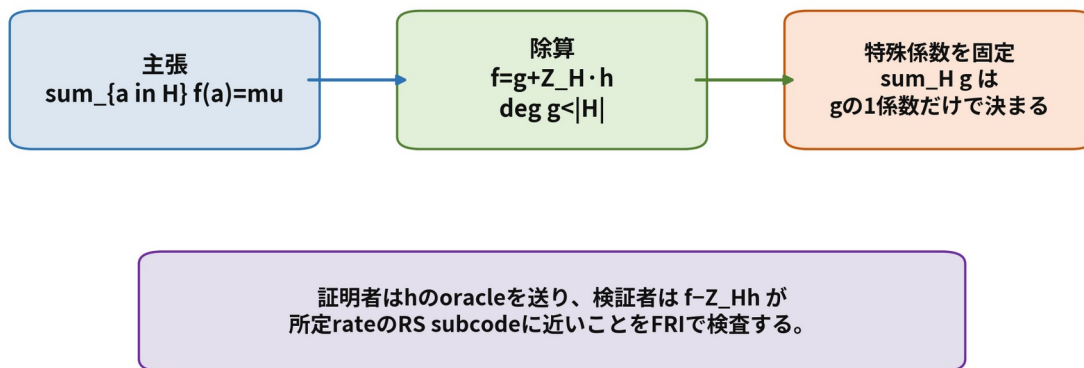


図 14 単変数 sumcheck の核心

本書作成。

12.2 multiplicative coset 版の恒等式

位数 m の乗法部分群 H について、 $\deg g < m$ なら $\sum_{a \in H} g(a) = m \cdot g(0)$ である。非定数 monomial X^k ($0 < k < m$) の和が 0 だからである。 $\deg f$ が m 以上でも、 $f = g + Z_H h$ と割れば H 上の値と sum は g に一致する。

$$f(X) = g(X) + Z_H(X)h(X), \quad \deg g < |H| \quad (12.1)$$

$$\sum_{a \in H} f(a) = \sum_{a \in H} g(a) = |H| \cdot g(0) \quad (12.2)$$

toy lincheck では $q(H) = (2, 4, 15, 13)$ 、sum は $34 \equiv 0$ である。補間すると $q(X) = 10X + 9X^3$ となり、定数項 $q(0) = 0$ である。したがって $\sum_H q = 4q(0) = 0$ である。

12.3 additive subspace 版

Aurora 本文の主要説明は二進拡大体の affine subspace を用いる。この場合、 $\deg g < |H|$ の sum は $X^{|H|-1}$ 係数だけで決まる Byott-Chapman 型恒等式を使う。multiplicative 版と「どの係数が sum を担うか」は異なるが、sum claim を特殊な RS subcode へ変換する構造は同じである。

12.4 amortization

三つの lincheck が生成する sum claims を個別に証明すると、sumcheck oracle と FFT が三倍になる。verifier は random weights z_A, z_B, z_C を選び、三 claim の linear combination を一つの sum claim へ束ねる。少なくとも一つが偽なら、combined residual が 0 になる確率は $1/|F|$ である。

12.5 RS-encoded IOP compiler

core protocol は、prover messages が指定 rate の RS codeword であると仮定し、rational constraints だけを出力する。この抽象化が RS-encoded IOP である。generic compiler は、全 oracles と virtual ratios を random linear combination し、FRI proximity instances へ落とす。この分離により、R1CS 代数を設計する層と、malformed oracle を排除する低次数検査層を独立に分析できる。libiop はこの分離を software component として実装した。

12.6 zero knowledge self-reduction

randomized LDE だけでは、sumcheck division 後に mask が消える。prover は random q と $\beta = \sum Hq$ を commit し、challenge c 後に $p = cf + q$ の sum target $c\mu + \beta$ を証明する。 p はほぼ uniform RS codeword であり、sumcheck transcript を witness から独立に simulate できる。

$$\sum H(cf + q) = c\mu + \beta, \quad \beta = \sum Hq \quad (12.3)$$

soundness 元 claim の sum が $a \neq \mu$ なら、 $ca + \beta = c\mu + \beta$ となる c は高々一つである。uniform c に対する失敗確率は $1/|F|$ である。

13. Aurora の実装・応用上の補足

統合論文との対応 統合論文 § 5.7-5.8、§ 6。

この章で得るもの explicit R1CS model の利点と限界、proof size の由来を評価できる。

Aurora は routing を避け、sparse matrices を直接 lincheck するため、explicit R1CS に対する IOP prover $O(N \log N)$ 、oracle length $O(N)$ を得る。query complexity $O(\log N)$ が BCS 変換後の短い argument へ直結する。

表 11 Aurora の適用条件

観点	Aurora が有利	Aurora が不利
statement representation	R1CS が既に存在し、matrix を verifier が読める	短い program の巨大実行を毎回 R1CS 展開する
proof size	透明・hash-based で polylog openings	pairing-based SNARK の数百 byte には及ばない
prover	routing/memory network を回避	sparse matrix scan と多数 FFT が必要
batch	同一 A,B,C を共有する複数 instance	matrix が毎回異なる

historical evidence Aurora 論文は 128-bit 設定、192-bit field で、数百万 constraints まで 40-130KB、proving 数分、verification 数秒を報告する[3, Abstract, Sec. 11]。同論文の 1M-gate 比較では Ligerio 4.0MB、STARK 3.2MB、Aurora 130KB と見積もるが、特定実装・特定 parameter の 2019 年時点比較である。

PART V

三方式を横断して設計判断する

toy computation、soundness、証拠の読み方、応用 mapping を一つに統合する。

14. F_17 toy computation の横断解剖

統合論文との対応 統合論文 § 3.8、§ 4.8、§ 5.9、付録 A。

この章で得るもの 同じ witness が三方式でどの情報構造へ変わるかを比較し、toy が省略する要素を明示する。

F_17 toy computationで観察すべき三つの「圧縮」

STARK
3遷移をtrace列へ
全制約をcompositionへ
次数をFRIで半減

Ligerio
8 wiresを 2×4 へ
多行をrandom combinationへ
全列からt列へ

Aurora
2 R1CS rowsへ
3 linchecksを1 sumcheckへ
多数oraclesをFRIへ

共通原理: 構造化された大域条件 + commit後の乱数 + 少数照会

図 15 三方式の toy で観察すべき圧縮

本書作成。

表 12 同一 relation の protocol 対応

段階	ZK-STARK	Ligerio	Aurora
semantic witness	$x=2$ 、逐次 accumulator	wire values 8 個	$x=2, t=12$
algebraic witness	4×3 trace	2×4 matrix	2-row R1CS
global consistency	transition/boundary constraints	linear/quadratic gate constraints	3 linchecks + rowcheck
random compression	constraint coefficients p	row/constraint coefficients r	α 、lincheck weights、sumcheck challenge
code check	FRI on composition/oracles	explicit RS check of combined word	FRI via generic compiler
ZK mask	Z_HR trace mask	randomized rows + u_0	randomized LDE + $cf+q$

14.1 toy が忠実に示すもの

- constraint values を polynomial identity へ変える操作。
- commit 後の random coefficients で多数条件を一つへ圧縮する操作。
- message domain 上の値を保ったまま、外部評価を randomize する操作。
- FRI folding や random column query が local consistency を確認する操作。

14.2 toy が省略するもの

- 128-bit 級 soundness amplification、複数 query、hash collision security。
- 実際の blowup factor、rate choice、grinding、extension field selection。
- STARK の APR/memory consistency、Aurora compiler の全 rational constraints。
- Merkle path pruning、batch opening、serialization、transcript domain separation。
- cache、parallelism、FFT plan、memory peak といった実装 cost。

検証済み数値 本書の F_17 補間、商、多項式除算、codeword 評価、FRI fold は mod 17 で機械検算している。付録 B に完全データを示す。

15. エビデンスに基づく比較と方式選択

統合論文との対応 統合論文 § 6-7、付録 B。

この章で得るもの asymptotic theorem、paper benchmark、cross-paper estimate を区別し、用途から方式を選ぶ。

15.1 information-theoretic IOP の漸近比較

表 13 explicit N-gate arithmetic circuit へ適用した IOP 比較

方式	round	oracle proof length	query	prover field ops	verifier field ops
Ligerio	2	$O(N)$	$O(\sqrt{N})$	$O(N \log N)$	$O(N)$
STARK	$O(\log N)$	$O(N \log N)$	$O(\log N)$	$O(N \log^2 N)$	$O(N)$
Aurora	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N \log N)$	$O(N)$

出典: Aurora [3, Fig. 1]。STARK は explicit circuit を bounded-halting problem へ変換した比較であり、native program representation 時の verifier benefit を表さない。

15.2 測値が食い違う理由

Aurora 2019 の 1M-gate 見積りでは Aurora が Ligerio より大幅に短いですが、Ligerio 2022 の改良実装表では小中規模で Ligerio-128 が Aurora 実装より短い例がある。これは矛盾ではない。security calibration、field size、statement generation、R1CS/circuit、implementation revision、Merkle batching、hardware が異なる。

表 14 ZKP 性能エビデンスの読み方

証拠型	信頼できる主張	してはいけない解釈
定理・漸近量	入力規模が十分大きいときの成長率	具体的 crossover を定数なしに断定
同一論文内 benchmark	同一 codebase/条件での相対差	他論文の別 hardware 値と直結
cross-paper estimate	rough order、設計傾向	厳密なランキング
toy computation	数式構造、message flow	security・performance の推定

15.3 selection matrix

方式選択の概念的decision tree

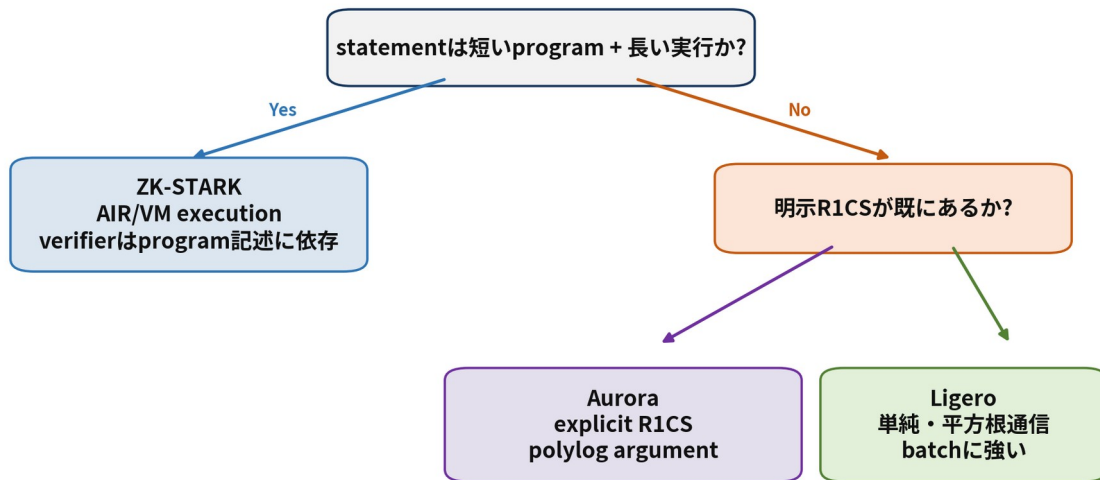


図 16 statement model から見た方式選択

本書作成。

表 15 応用別の概念的 mapping

use case	第一候補	理由	注意
長い VM/逐次 state transition	ZK-STARK	program が短く T が長い uniform computation	AIR/memory arithmetization の質が支配
既存 R1CS ecosystem	Aurora	explicit matrices を直接扱い、短い transparent argument	verifier は matrix size に線形
中規模・単純実装・batch	Ligerio	2-round core、sqrt communication、amortization	大規模では polylog proof に漸近的に負ける
秘匿 database scan	ZK-STARK / Ligerio	逐次 scan なら STARK、同一回路多数なら Ligerio	data commitment との binding が必要
同一 business logic の多数 instance	Ligerio / Aurora	matrix/circuit 共有による amortization	instance-specific public input 処理

16. soundness と security review の実務

統合論文との対応 統合論文 § 2.3-2.4、§ 6.3、§ 7.5。

この章で得るもの soundness error を層別に分解し、実装監査の質問へ変換する。

最終 soundness は単一の「 $2^{-\lambda}$ 」という箱ではない。arithmetization error、random aggregation、proximity test、sampling、Merkle binding、Fiat-Shamir transform の損失を合成する。union bound を使う場合、各項へ security budget を配分する。

表 16 soundness/security の層別チェック

層	典型的 error	監査質問
algebraic aggregation	$d/ F , 1/ F $	challenge は commit 後か。field は十分大きいかな。
sampling	$(1-\delta)^t$	query は独立か。重複処理は正しいかな。
FRI/LDT	rate · distance 依存	round 数、blowup、query count は analysis と一致するか。
Merkle	hash collision	leaf encoding は prefix-free か。path index は binding されるか。
Fiat-Shamir	ROM/QROM reduction loss	domain separation、transcript order、grinding を扱うか。
zero knowledge	query bound / simulator	reused randomness、domain overlap、mask degree 不足がないかな。

実装上の重大点 prover と verifier が同じ field element serialization、domain ordering、padding rule、vanishing polynomial convention を共有しなければならない。数学的 protocol が正しくても、encoding の曖昧さは別の attack surface となる。

17. よくある難所と誤解の解消

統合論文との対応 統合論文全体。

この章で得るもの 読解時に混同しやすい概念を短い Q&A で固定する。

Q1. 低次数なら、なぜ計算が正しいのか。 低次数性だけでは足りない。計算 constraints から作った composition/ratio が低次数であること、元 oracle と同じ commitment 位置で整合すること、境界条件が成立することを組み合わせる。

Q2. proof oracle が $O(N)$ なら succinct ではないのか。 IOP model では oracle 全体を概念的に送るが、argument では root と少数 opening だけを送る。succinctness は final communication と verification に対する性質である。

Q3. random linear combination で全 constraint を本当に検査できるか。 少なくとも一つの residual が非零なら、random coefficient で和が 0 になるのは field 上の低確率 event である。commit 前に係数を知ると、この議論は壊れる。

Q4. FRI は error-correcting decode をするのか。 通常の verifier は完全 decode しない。folding と少数 query で、far word が accept されにくいことを確認する。knowledge extraction の proof では decoding argument が使われる。

Q5. randomized LDE は witness を変更しないか。 H 上の値は Z_{HR} が 0 なので変わらない。変更されるのは H 外の冗長評価だけである。

Q6. STARK と Aurora はどちらも FRI なら同じ方式か。 backend は共有するが、arithmetization と core IOP が異なる。STARK は execution trace/AIR、Aurora は explicit R1CS/lincheck/sumcheck である。

Q7. Ligerio に FRI を追加すれば Aurora になるか。 ならない。Aurora の log query は univariate sumcheck により unstructured linear constraints を圧縮する点にあり、単に LDT を置換するだけでは sqrt column openings は消えない。

18. 結論

三方式の共通本質は、計算を有限体上の低次数関係へ変換し、Reed-Solomon 冗長性と commit 後の乱数により、少数照会へ圧縮することである。相違は、どの計算表現を最初に選び、どの検査を低次数 backend へ渡すかにある。

ZK-STARK は uniform execution を trace として扱い、Ligerio は witness を平方行列化して簡潔な interleaved tests を行い、Aurora は explicit R1CS を lincheck・rowcheck・univariate sumcheck へ分解する。零知識化は、いずれも「message を固定する多項式の affine space」を利用するが、開示する対象に応じて trace mask、blind codeword、self-reduction へ分化する。

実務上は、漸近式だけで方式を選ばず、statement representation、security calibration、field/FFT、memory、hash、batching、verifier が読む instance size を含む end-to-end cost で比較すべきである。本書の F_17 toy は、この設計判断を支える最小の計算モデルである。

付録 A. F_{17} arithmetic と domain 一覧

表 A1 toy で用いる domain

集合	要素	性質
F_{17}	$0, 1, \dots, 16$	prime field
F_{17}^*	$1, \dots, 16$	order 16 cyclic group
H_4	$\{1, 4, 16, 13\}$	$4^4=1$, trace/R1CS constraint domain
D_8	$\{1, 2, 4, 8, 16, 15, 13, 9\}$	$2^8=1$, toy FRI domain
$3D_8$	$\{3, 6, 12, 7, 14, 11, 5, 10\}$	D_8 の coset, H_4 と disjoint
ζ	$\{0, 1, 2, 3\}$	Ligerio message points
η	$\{4, 5, 6, 7, 8, 9, 10, 11\}$	Ligerio codeword points

$$Z_{\{H_4\}}(X) = X^4 - 1 \quad (\text{A.1})$$

$$Z_{\zeta}(X) = X(X-1)(X-2)(X-3) = X^4 + 11X^3 + 11X^2 + 11X \quad (\text{A.2})$$

付録 B. toy computation 完全データ

B.1 STARK

表 B1 STARK trace/domain evaluations

T	X(T)	I(T)	A(T)	D(T) on D_8
1	2	1	1	0
2	-	-	-	6
4	2	2	3	1
8	-	-	-	4
16	2	3	12	13
15	-	-	-	8
13	2	4	9	0
9	-	-	-	13

$$X(T)=2 \quad (\text{B.1})$$

$$I(T)=11+10T+8T^2+6T^3 \quad (\text{B.2})$$

$$A(T)=2+16T+13T^2+4T^3 \quad (\text{B.3})$$

B.2 Ligerio

表 B2 Ligerio toy polynomials

polynomial	expression mod 17
p_1	1+X
p_2	5+2X+10X^2+12X^3
p~_1	1+11X+15X^2+15X^3+9X^4+2X^5
p~_2	5+11X+6X^2+8X^3+11X^4+5X^5
u_0	$Z_\zeta(6+3X)$
quadratic quotient	$3X^2+11X+5$

B.3 Aurora

表 B3 Aurora toy polynomials

oracle	masked polynomial
f_z	$4+4X+4X^2+X^3+2X^4+3X^5$
f_A	$4+9X+2X^2+4X^4+X^5$
f_B	$8+8X+4X^2+6X^3+7X^4+5X^5$
f_C	$12+9X+5X^2+12X^3+6X^4+2X^5$
q_{row}	$14+7X+3X^2+4X^3+10X^5+5X^6$
q_{lin}	$10X+9X^3$

付録 C. 用語集

表 C1 用語集

用語	本書での意味
AIR	Algebraic Intermediate Representation。current/next state の低次数 constraints で実行を表す。
APR	Algebraic Placement and Routing。一般 memory access を代数的 graph へ配置・routing する。
ALI	Algebraic Linking IOP。多数 AIR constraints を random combination し、RS proximity 問題へ接続する。
LDE	Low-Degree Extension。同じ補間多項式を大きな domain で評価する。
FRI	Fast Reed-Solomon IOP of Proximity。folding で degree を反復的に下げる。
interleaved code	複数 codeword を row に持つ matrix code。Ligerio では distance を列単位で測る。
rowcheck	entry-wise product など、各 constraint row の局所代数関係を検査する。
lincheck	$x=My$ のような線形変換を random power vector と sumcheck で検査する。
univariate sumcheck	単変数 polynomial の domain sum を、division と低次数 subcode へ帰着する。
rational constraint	既存 oracles から比 N/D として virtual oracle を定義し、rate 条件を課す。
bounded independence	限られた個数の照会値が一様独立に見える性質。randomized LDE の ZK に用いる。
self-reduction	元 claim を randomized claim へ変換し、witness 依存 transcript を消す。
BCS transformation	public-coin IOP を Merkle commitment と random oracle で non-interactive argument へ変換する枠組み。

付録 D. 参考文献とエビデンス注記

- [1] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” 2018.
- [2] S. Ames, C. Hazay, Y. Ishai, M. Venkatasubramanian, “Ligerio: Lightweight Sublinear Arguments Without a Trusted Setup,” extended version, IACR ePrint 2022/1608.
- [3] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, N. P. Ward, “Aurora: Transparent Succinct Arguments for RICS,” 2019.
- [4] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” 2017.
- [5] E. Ben-Sasson, A. Chiesa, N. Spooner, “Interactive Oracle Proofs,” TCC 2016.
- [6] C. Lund, L. Fortnow, H. Karloff, N. Nisan, “Algebraic Methods for Interactive Proof Systems,” JACM, 1992.
- [7] N. Byott, R. Chapman, finite-field power-sum identity used by Aurora, 1999.
- [8] S.-J. Lin, W.-H. Chung, Y. S. Han, additive FFT / polynomial basis work, FOCS 2014.
- [9] R. Zippel, “Probabilistic algorithms for sparse polynomials,” 1979; J. T. Schwartz, “Fast probabilistic algorithms for verification of polynomial identities,” 1980.
- [10] R. C. Merkle, authenticated hash trees; A. Fiat, A. Shamir, public-coin to non-interactive heuristic.

比較上の原則 異なる論文の proof size ・ time を引用する場合、security level、field、statement model、hardware、preprocessing、serialization を確認する。本書は原論文の主張を再掲するが、現在の best implementation を断定しない。